

forecasting

May 6, 2026

```
[1]: # =====  
# STEP 3 - SALES FORECASTING  
# Tool: Python (Prophet)  
# =====  
  
import pandas as pd  
import matplotlib.pyplot as plt  
from prophet import Prophet  
import warnings  
warnings.filterwarnings('ignore')  
  
print(" Libraries imported successfully!")
```

```
C:\Users\TUF\PyCharmMiscProject\.venv\Lib\site-packages\tqdm\auto.py:21:  
TqdmWarning: IProgress not found. Please update jupyter and ipywidgets. See  
https://ipywidgets.readthedocs.io/en/stable/user_install.html  
    from .autonotebook import tqdm as notebook_tqdm  
Importing plotly failed. Interactive plots will not work.  
  
Libraries imported successfully!
```

```
[2]: # =====  
# LOAD DATA & PREPARE FOR PROPHET  
# =====  
  
# Load clean dataset  
df = pd.read_csv('../Clean_Superstore.csv')  
  
# Convert Order Date to datetime  
df['Order Date'] = pd.to_datetime(df['Order Date'])  
  
# Group sales by month  
monthly_sales = df.groupby(pd.Grouper(key='Order Date', freq='MS'))['Sales'].  
    ↪sum().reset_index()  
  
# Rename columns for Prophet (requires 'ds' and 'y')  
monthly_sales.columns = ['ds', 'y']
```

```

print(" Data prepared for forecasting!")
print(f"Total months of data: {len(monthly_sales)}")
print(f"\nSample data:")
print(monthly_sales.head(10))
print(f"\nDate range: {monthly_sales['ds'].min()} to {monthly_sales['ds'].
↪max()}")

```

Data prepared for forecasting!
Total months of data: 48

Sample data:

	ds	y
0	2014-01-01	17712.1660
1	2014-02-01	5708.2320
2	2014-03-01	56783.2050
3	2014-04-01	30832.9450
4	2014-05-01	25138.0610
5	2014-06-01	26592.7436
6	2014-07-01	33229.1340
7	2014-08-01	34644.2855
8	2014-09-01	62787.2458
9	2014-10-01	26313.3340

Date range: 2014-01-01 00:00:00 to 2017-12-01 00:00:00

```

[3]: # =====
# TRAIN THE PROPHET MODEL
# =====

print("Training Prophet model...")

# Create and train the model
model = Prophet(
    yearly_seasonality=True,    # detect yearly patterns
    weekly_seasonality=False,  # not needed for monthly data
    daily_seasonality=False,   # not needed for monthly data
    seasonality_mode='multiplicative' # better for growing trends
)

# Train the model on our data
model.fit(monthly_sales)

print(" Model trained successfully!")

```

16:45:15 - cmdstanpy - INFO - Chain [1] start processing

Training Prophet model...

16:45:15 - cmdstanpy - INFO - Chain [1] done processing

Model trained successfully!

```
[4]: # =====  
# MAKE FUTURE PREDICTIONS  
# =====  
  
# Create future dates - 12 months ahead  
future = model.make_future_dataframe(periods=12, freq='MS')  
  
print(f"Predicting for {len(future)} months total")  
print(f"Future dates preview:")  
print(future.tail(12))  
  
# Generate predictions  
forecast = model.predict(future)  
  
# Show key prediction columns  
print(f"\n Forecast generated!")  
print(f"\nForecast preview (last 12 months = predictions):")  
print(forecast[['ds', 'yhat', 'yhat_lower', 'yhat_upper']].tail(12))
```

Predicting for 60 months total

Future dates preview:

	ds
48	2018-01-01
49	2018-02-01
50	2018-03-01
51	2018-04-01
52	2018-05-01
53	2018-06-01
54	2018-07-01
55	2018-08-01
56	2018-09-01
57	2018-10-01
58	2018-11-01
59	2018-12-01

Forecast generated!

Forecast preview (last 12 months = predictions):

	ds	yhat	yhat_lower	yhat_upper
48	2018-01-01	37492.737545	26708.933852	48349.534138
49	2018-02-01	28621.365150	17973.360852	39391.024698
50	2018-03-01	74062.432265	63019.330209	85184.406878
51	2018-04-01	47590.066190	35277.708128	58427.557836
52	2018-05-01	45683.270466	34197.774988	57148.178741
53	2018-06-01	46638.314659	35116.228997	57205.519367
54	2018-07-01	51085.014355	40373.393823	62852.711111

55	2018-08-01	66754.638829	55912.453094	77955.094827
56	2018-09-01	90422.721880	80276.985977	102041.753462
57	2018-10-01	58253.494336	46735.679958	68911.539961
58	2018-11-01	102269.403616	91402.664074	113262.629200
59	2018-12-01	85997.551313	74917.957445	96722.619356

```
[5]: # =====
# PLOT THE FORECAST
# =====

fig, ax = plt.subplots(figsize=(14, 6))

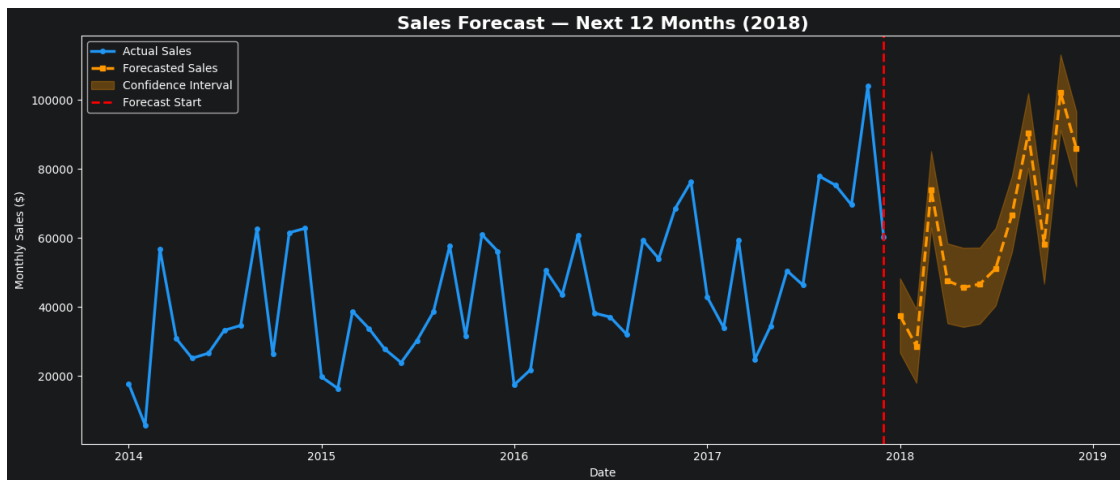
# Plot actual historical data
ax.plot(monthly_sales['ds'], monthly_sales['y'],
        color='#2196F3', linewidth=2.5,
        marker='o', markersize=4, label='Actual Sales')

# Plot predicted values
forecast_future = forecast[forecast['ds'] > monthly_sales['ds'].max()]
ax.plot(forecast_future['ds'], forecast_future['yhat'],
        color='#FF9800', linewidth=2.5,
        marker='s', markersize=4, linestyle='--', label='Forecasted Sales')

# Plot confidence interval
ax.fill_between(forecast_future['ds'],
               forecast_future['yhat_lower'],
               forecast_future['yhat_upper'],
               alpha=0.3, color='#FF9800', label='Confidence Interval')

# Add vertical line separating actual vs forecast
ax.axvline(x=monthly_sales['ds'].max(),
           color='red', linewidth=2,
           linestyle='--', label='Forecast Start')

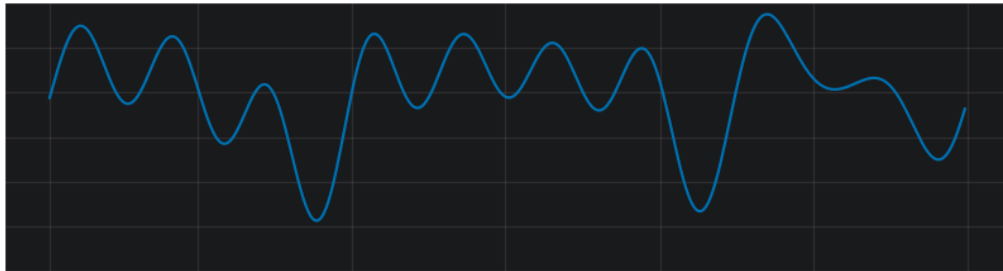
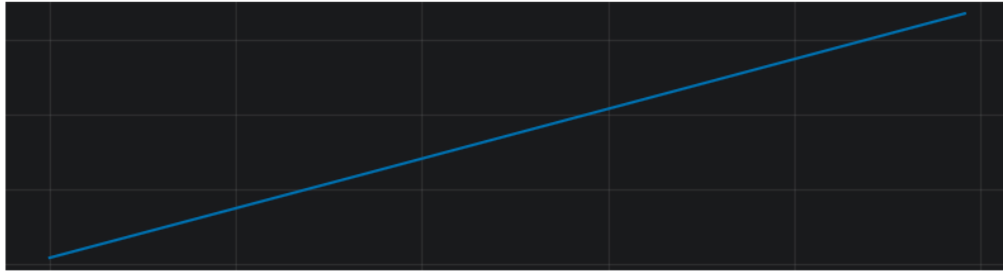
ax.set_title('Sales Forecast - Next 12 Months (2018)',
             fontsize=16, fontweight='bold')
ax.set_xlabel('Date')
ax.set_ylabel('Monthly Sales ($)')
ax.legend()
plt.tight_layout()
plt.savefig('chart9_forecast.png', dpi=150, bbox_inches='tight')
plt.show()
print(" Forecast chart saved as 'chart9_forecast.png'")
```



Forecast chart saved as 'chart9_forecast.png'

```
[6]: # =====
# PLOT SEASONAL COMPONENTS
# =====

fig2 = model.plot_components(forecast)
plt.suptitle('Sales Forecast Components', fontsize=14, fontweight='bold')
plt.tight_layout()
plt.savefig('chart10_forecast_components.png', dpi=150, bbox_inches='tight')
plt.show()
print(" Components chart saved as 'chart10_forecast_components.png'")
```



Components chart saved as 'chart10_forecast_components.png'

```
[7]: # =====  
# STEP 4 - ADVANCED ANALYSIS  
# =====  
  
import pandas as pd  
import matplotlib.pyplot as plt  
import seaborn as sns  
import numpy as np  
  
# Settings  
plt.rcParams['figure.figsize'] = (12, 6)  
plt.rcParams['font.size'] = 12  
sns.set_theme(style='whitegrid')  
  
# Load clean dataset  
df = pd.read_csv('../Clean_Superstore.csv')  
  
# Convert dates  
df['Order Date'] = pd.to_datetime(df['Order Date'])  
df['Ship Date'] = pd.to_datetime(df['Ship Date'])
```

```
print(" Dataset loaded successfully!")
print(f"Shape: {df.shape}")
```

Dataset loaded successfully!
Shape: (9733, 25)

```
[8]: # =====
# ANALYSIS 1 - PROFIT MARGIN BY CATEGORY & REGION
# =====

print("=" * 50)
print("ANALYSIS 1 - PROFIT MARGIN ANALYSIS")
print("=" * 50)

# Calculate profit margin %
df['Profit Margin %'] = (df['Profit'] / df['Sales']) * 100

# Profit margin by Category
category_margin = df.groupby('Category')['Profit Margin %'].mean().reset_index()
category_margin = category_margin.sort_values('Profit Margin %',
    ↪ascending=False)
print(f"\nProfit Margin by Category:")
print(category_margin)

# Profit margin by Region
region_margin = df.groupby('Region')['Profit Margin %'].mean().reset_index()
region_margin = region_margin.sort_values('Profit Margin %', ascending=False)
print(f"\nProfit Margin by Region:")
print(region_margin)

# Profit margin by Sub-Category
subcategory_margin = df.groupby('Sub-Category')['Profit Margin %'].mean().
    ↪reset_index()
subcategory_margin = subcategory_margin.sort_values('Profit Margin %',
    ↪ascending=False)

# Create charts
fig, axes = plt.subplots(1, 3, figsize=(18, 6))

# Category margin
colors1 = ['#4CAF50' if x > 0 else '#F44336' for x in category_margin['Profit_
    ↪Margin %']]
axes[0].barh(category_margin['Category'], category_margin['Profit Margin %'],
    ↪color=colors1)
axes[0].axvline(x=0, color='black', linewidth=1.5)
axes[0].set_title('Avg Profit Margin % by Category', fontweight='bold')
axes[0].set_xlabel('Profit Margin %')
```

```

for i, v in enumerate(category_margin['Profit Margin %']):
    axes[0].text(v + 0.5, i, f'{v:.1f}%', va='center', fontweight='bold')

# Region margin
colors2 = ['#4CAF50' if x > 0 else '#F44336' for x in region_margin['Profit_
    ↪Margin %']]
axes[1].barh(region_margin['Region'], region_margin['Profit Margin %'],
    ↪color=colors2)
axes[1].axvline(x=0, color='black', linewidth=1.5)
axes[1].set_title('Avg Profit Margin % by Region', fontweight='bold')
axes[1].set_xlabel('Profit Margin %')
for i, v in enumerate(region_margin['Profit Margin %']):
    axes[1].text(v + 0.5, i, f'{v:.1f}%', va='center', fontweight='bold')

# Sub-category margin
colors3 = ['#4CAF50' if x > 0 else '#F44336' for x in
    ↪subcategory_margin['Profit Margin %']]
axes[2].barh(subcategory_margin['Sub-Category'], subcategory_margin['Profit_
    ↪Margin %'], color=colors3)
axes[2].axvline(x=0, color='black', linewidth=1.5)
axes[2].set_title('Avg Profit Margin % by Sub-Category', fontweight='bold')
axes[2].set_xlabel('Profit Margin %')

plt.suptitle('Profit Margin Analysis', fontsize=16, fontweight='bold')
plt.tight_layout()
plt.savefig('chart11_profit_margin.png', dpi=150, bbox_inches='tight')
plt.show()
print(" Chart saved as 'chart11_profit_margin.png'")

```

```

=====
ANALYSIS 1 - PROFIT MARGIN ANALYSIS
=====

```

Profit Margin by Category:

	Category	Profit Margin %
2	Technology	24.693123
1	Office Supplies	17.754103
0	Furniture	3.912054

Profit Margin by Region:

	Region	Profit Margin %
3	West	23.608523
1	East	19.951104
2	South	18.871892
0	Central	-1.734658

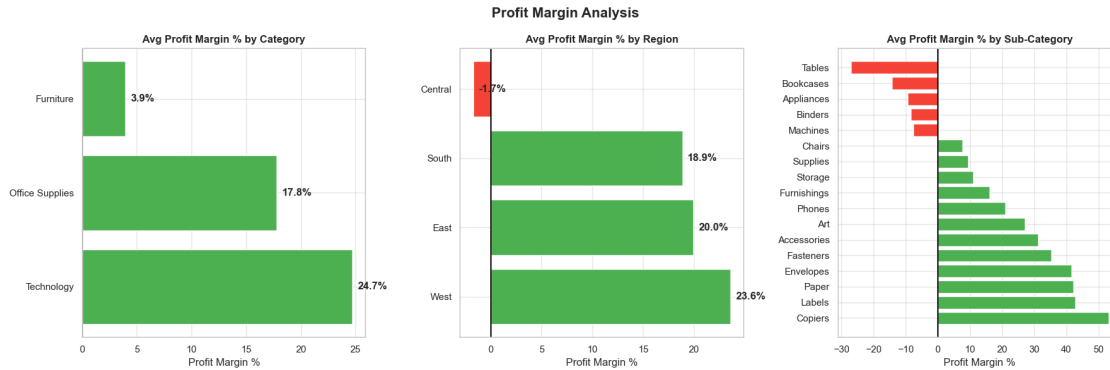


Chart saved as 'chart11_profit_margin.png'

```
[9]: # =====
# ANALYSIS 2 - SHIPPING DURATION ANALYSIS
# =====

print("=" * 50)
print("ANALYSIS 2 - SHIPPING DURATION ANALYSIS")
print("=" * 50)

# Calculate shipping duration in days
df['Shipping Days'] = (df['Ship Date'] - df['Order Date']).dt.days

# Remove negative or zero shipping days (data errors)
df = df[df['Shipping Days'] >= 0]

# Average shipping days by Ship Mode
shipmode_days = df.groupby('Ship Mode')['Shipping Days'].mean().reset_index()
shipmode_days = shipmode_days.sort_values('Shipping Days')
print(f"\nAverage Shipping Days by Ship Mode:")
print(shipmode_days)

# Average shipping days by Region
region_days = df.groupby('Region')['Shipping Days'].mean().reset_index()
region_days = region_days.sort_values('Shipping Days')
print(f"\nAverage Shipping Days by Region:")
print(region_days)

# Create charts
fig, axes = plt.subplots(1, 3, figsize=(18, 6))

# Ship Mode shipping days
colors = ['#4CAF50', '#2196F3', '#FF9800', '#F44336']
bars = axes[0].barh(shipmode_days['Ship Mode'],
```

```

        shipmode_days['Shipping Days'],
        color=colors)
axes[0].set_title('Avg Shipping Days by Ship Mode', fontweight='bold')
axes[0].set_xlabel('Average Days')
for i, v in enumerate(shipmode_days['Shipping Days']):
    axes[0].text(v + 0.05, i, f'{v:.1f} days', va='center', fontweight='bold')

# Region shipping days
axes[1].barh(region_days['Region'],
            region_days['Shipping Days'],
            color='#9C27B0')
axes[1].set_title('Avg Shipping Days by Region', fontweight='bold')
axes[1].set_xlabel('Average Days')
for i, v in enumerate(region_days['Shipping Days']):
    axes[1].text(v + 0.05, i, f'{v:.1f} days', va='center', fontweight='bold')

# Distribution of shipping days
axes[2].hist(df['Shipping Days'], bins=15,
            color='#2196F3', edgecolor='white')
axes[2].set_title('Distribution of Shipping Days', fontweight='bold')
axes[2].set_xlabel('Shipping Days')
axes[2].set_ylabel('Number of Orders')

plt.suptitle('Shipping Duration Analysis', fontsize=16, fontweight='bold')
plt.tight_layout()
plt.savefig('chart12_shipping.png', dpi=150, bbox_inches='tight')
plt.show()
print(" Chart saved as 'chart12_shipping.png'")

```

```

=====
ANALYSIS 2 - SHIPPING DURATION ANALYSIS
=====

```

Average Shipping Days by Ship Mode:

	Ship Mode	Shipping Days
1	Same Day	13.049383
0	First Class	17.389481
2	Second Class	17.960137
3	Standard Class	18.706633

Average Shipping Days by Region:

	Region	Shipping Days
2	South	16.948700
0	Central	17.019542
3	West	18.429022
1	East	19.106307

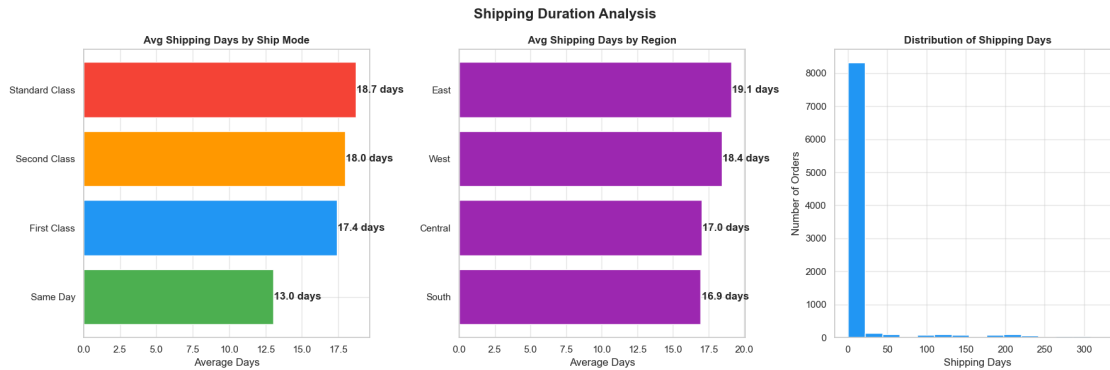


Chart saved as 'chart12_shipping.png'

```
[11]: # =====
# FIX - FILTER REALISTIC SHIPPING DAYS
# =====

print("Shipping days distribution BEFORE filter:")
print(df['Shipping Days'].describe())

# Keep only realistic shipping days (0-30 days)
df_ship = df[df['Shipping Days'] <= 30].copy()

print(f"\nRows removed: {len(df) - len(df_ship)}")
print(f"\nShipping days distribution AFTER filter:")
print(df_ship['Shipping Days'].describe())

# Recalculate with clean data
shipmode_days = df_ship.groupby('Ship Mode')['Shipping Days'].mean().
    reset_index()
shipmode_days = shipmode_days.sort_values('Shipping Days')
region_days = df_ship.groupby('Region')['Shipping Days'].mean().reset_index()
region_days = region_days.sort_values('Shipping Days')

print(f"\nCorrected Avg Shipping Days by Ship Mode:")
print(shipmode_days)
print(f"\nCorrected Avg Shipping Days by Region:")
print(region_days)

# Recreate charts with clean data
fig, axes = plt.subplots(1, 3, figsize=(18, 6))

colors = ['#4CAF50', '#2196F3', '#FF9800', '#F44336']
axes[0].barh(shipmode_days['Ship Mode'],
              shipmode_days['Shipping Days'], color=colors)
```

```

axes[0].set_title('Avg Shipping Days by Ship Mode', fontweight='bold')
axes[0].set_xlabel('Average Days')
for i, v in enumerate(shipmode_days['Shipping Days']):
    axes[0].text(v + 0.05, i, f'{v:.1f} days', va='center', fontweight='bold')

axes[1].barh(region_days['Region'],
             region_days['Shipping Days'], color='#9C27B0')
axes[1].set_title('Avg Shipping Days by Region', fontweight='bold')
axes[1].set_xlabel('Average Days')
for i, v in enumerate(region_days['Shipping Days']):
    axes[1].text(v + 0.05, i, f'{v:.1f} days', va='center', fontweight='bold')

axes[2].hist(df_ship['Shipping Days'], bins=15,
             color='#2196F3', edgecolor='white')
axes[2].set_title('Distribution of Shipping Days', fontweight='bold')
axes[2].set_xlabel('Shipping Days')
axes[2].set_ylabel('Number of Orders')

plt.suptitle('Shipping Duration Analysis (Corrected)', fontsize=16,
            fontweight='bold')
plt.tight_layout()
plt.savefig('chart12_shipping.png', dpi=150, bbox_inches='tight')
plt.show()
print(" Corrected chart saved!")

```

Shipping days distribution BEFORE filter:

```

count    9212.000000
mean      18.064698
std       50.716558
min        0.000000
25%        3.000000
50%        4.000000
75%        6.000000
max       330.000000

```

Name: Shipping Days, dtype: float64

Rows removed: 886

Shipping days distribution AFTER filter:

```

count    8326.000000
mean      4.004924
std       2.042456
min        0.000000
25%        3.000000
50%        4.000000
75%        5.000000
max       29.000000

```

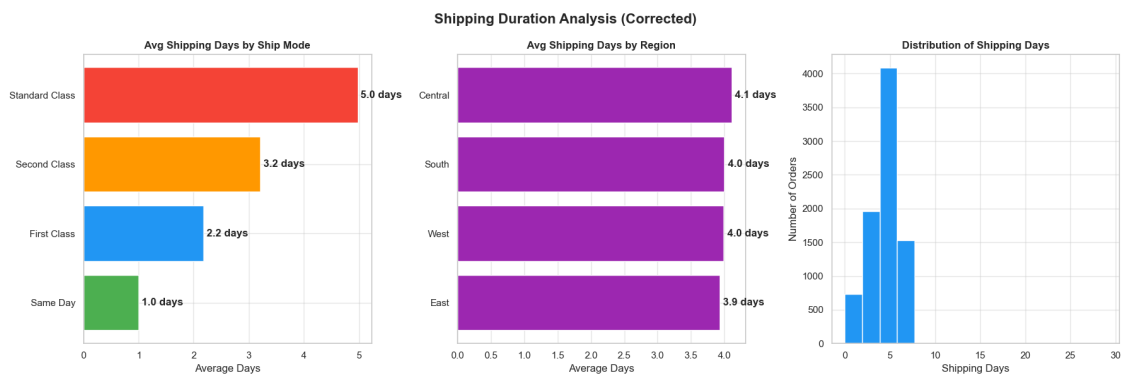
Name: Shipping Days, dtype: float64

Corrected Avg Shipping Days by Ship Mode:

	Ship Mode	Shipping Days
1	Same Day	1.000000
0	First Class	2.181314
2	Second Class	3.210726
3	Standard Class	4.979924

Corrected Avg Shipping Days by Region:

	Region	Shipping Days
1	East	3.931628
3	West	3.992650
2	South	4.000771
0	Central	4.113386



Corrected chart saved!

```
[13]: # =====
# ANALYSIS 3 - RFM CUSTOMER SEGMENTATION
# =====

print("=" * 50)
print("ANALYSIS 3 - RFM ANALYSIS")
print("=" * 50)

# Set reference date (day after last order)
reference_date = df['Order Date'].max() + pd.Timedelta(days=1)
print(f"Reference date: {reference_date}")

# Calculate RFM metrics per customer
rfm = df.groupby('Customer Name').agg(
    Recency=('Order Date', lambda x: (reference_date - x.max()).days),
    Frequency=('Order ID', 'count'),
    Monetary=('Sales', 'sum')
```

```

).reset_index()

print(f"\nRFM Sample:")
print(rfm.head(10))
print(f"\nRFM Statistics:")
print(rfm[['Recency', 'Frequency', 'Monetary']].describe())

```

```

=====
ANALYSIS 3 - RFM ANALYSIS
=====
Reference date: 2017-12-31 00:00:00

```

RFM Sample:

	Customer Name	Recency	Frequency	Monetary
0	Aaron Bergman	446	4	819.500
1	Aaron Hawkins	13	11	1476.286
2	Aaron Smayling	118	8	2326.140
3	Adam Bellavance	55	17	7587.680
4	Adam Hart	69	19	3302.235
5	Adam Shillingsburg	29	22	3241.060
6	Adrian Barton	42	20	4635.517
7	Adrian Hane	61	16	1687.270
8	Adrian Shami	42	3	58.820
9	Aimee Bixby	42	8	271.680

RFM Statistics:

	Recency	Frequency	Monetary
count	792.000000	792.000000	792.000000
mean	160.680556	11.631313	2558.921433
std	198.047569	12.143492	3400.585095
min	1.000000	1.000000	4.833000
25%	37.000000	7.000000	908.057750
50%	90.000000	11.000000	1866.778000
75%	197.000000	14.000000	3184.979750
max	1192.000000	313.000000	70056.567200

```

[14]: # =====
# SCORE & SEGMENT CUSTOMERS
# =====

# Score each metric 1-4 (4 = best)
rfm['R_Score'] = pd.qcut(rfm['Recency'], q=4, labels=[4, 3, 2, 1])
rfm['F_Score'] = pd.qcut(rfm['Frequency'].rank(method='first'), q=4, labels=[1, 2, 3, 4])
rfm['M_Score'] = pd.qcut(rfm['Monetary'].rank(method='first'), q=4, labels=[1, 2, 3, 4])

```

```

# Convert to int
rfm['R_Score'] = rfm['R_Score'].astype(int)
rfm['F_Score'] = rfm['F_Score'].astype(int)
rfm['M_Score'] = rfm['M_Score'].astype(int)

# Calculate total RFM score
rfm['RFM_Score'] = rfm['R_Score'] + rfm['F_Score'] + rfm['M_Score']

# Segment customers based on RFM score
def segment_customer(score):
    if score >= 10:
        return 'Champions'
    elif score >= 8:
        return 'Loyal Customers'
    elif score >= 6:
        return 'Potential Loyalists'
    elif score >= 4:
        return 'At Risk'
    else:
        return 'Lost Customers'

rfm['Segment'] = rfm['RFM_Score'].apply(segment_customer)

print("Customer Segments:")
print(rfm['Segment'].value_counts())
print(f"\nSample of segmented customers:")
print(rfm[['Customer Name', 'Recency', 'Frequency', 'Monetary', 'RFM_Score', 'Segment']].head(10))

```

Customer Segments:

Segment

Potential Loyalists 209

Champions 206

Loyal Customers 198

At Risk 115

Lost Customers 64

Name: count, dtype: int64

Sample of segmented customers:

	Customer Name	Recency	Frequency	Monetary	RFM_Score	\
0	Aaron Bergman	446	4	819.500	3	
1	Aaron Hawkins	13	11	1476.286	8	
2	Aaron Smayling	118	8	2326.140	7	
3	Adam Bellavance	55	17	7587.680	11	
4	Adam Hart	69	19	3302.235	11	
5	Adam Shillingsburg	29	22	3241.060	12	
6	Adrian Barton	42	20	4635.517	11	
7	Adrian Hane	61	16	1687.270	9	

8	Adrian Shami	42	3	58.820	5
9	Aimee Bixby	42	8	271.680	6

	Segment
0	Lost Customers
1	Loyal Customers
2	Potential Loyalists
3	Champions
4	Champions
5	Champions
6	Champions
7	Loyal Customers
8	At Risk
9	Potential Loyalists

```
[15]: # =====
# RFM VISUALIZATION
# =====

fig, axes = plt.subplots(2, 2, figsize=(16, 12))

# Chart 1 - Customer Segments Count
segment_counts = rfm['Segment'].value_counts()
colors = ['#4CAF50', '#2196F3', '#FF9800', '#F44336', '#9C27B0']
axes[0, 0].bar(segment_counts.index, segment_counts.values, color=colors)
axes[0, 0].set_title('Customer Segments Distribution', fontweight='bold')
axes[0, 0].set_xlabel('Segment')
axes[0, 0].set_ylabel('Number of Customers')
axes[0, 0].tick_params(axis='x', rotation=15)
for i, v in enumerate(segment_counts.values):
    axes[0, 0].text(i, v + 2, str(v), ha='center', fontweight='bold')

# Chart 2 - Average Monetary by Segment
segment_monetary = rfm.groupby('Segment')['Monetary'].mean().
    ↪sort_values(ascending=False)
axes[0, 1].bar(segment_monetary.index, segment_monetary.values, color=colors)
axes[0, 1].set_title('Average Spending by Segment', fontweight='bold')
axes[0, 1].set_xlabel('Segment')
axes[0, 1].set_ylabel('Average Sales ($)')
axes[0, 1].tick_params(axis='x', rotation=15)
for i, v in enumerate(segment_monetary.values):
    axes[0, 1].text(i, v + 20, f'${v:,.0f}', ha='center', fontweight='bold',
    ↪fontsize=9)

# Chart 3 - Average Frequency by Segment
segment_freq = rfm.groupby('Segment')['Frequency'].mean().
    ↪sort_values(ascending=False)
```



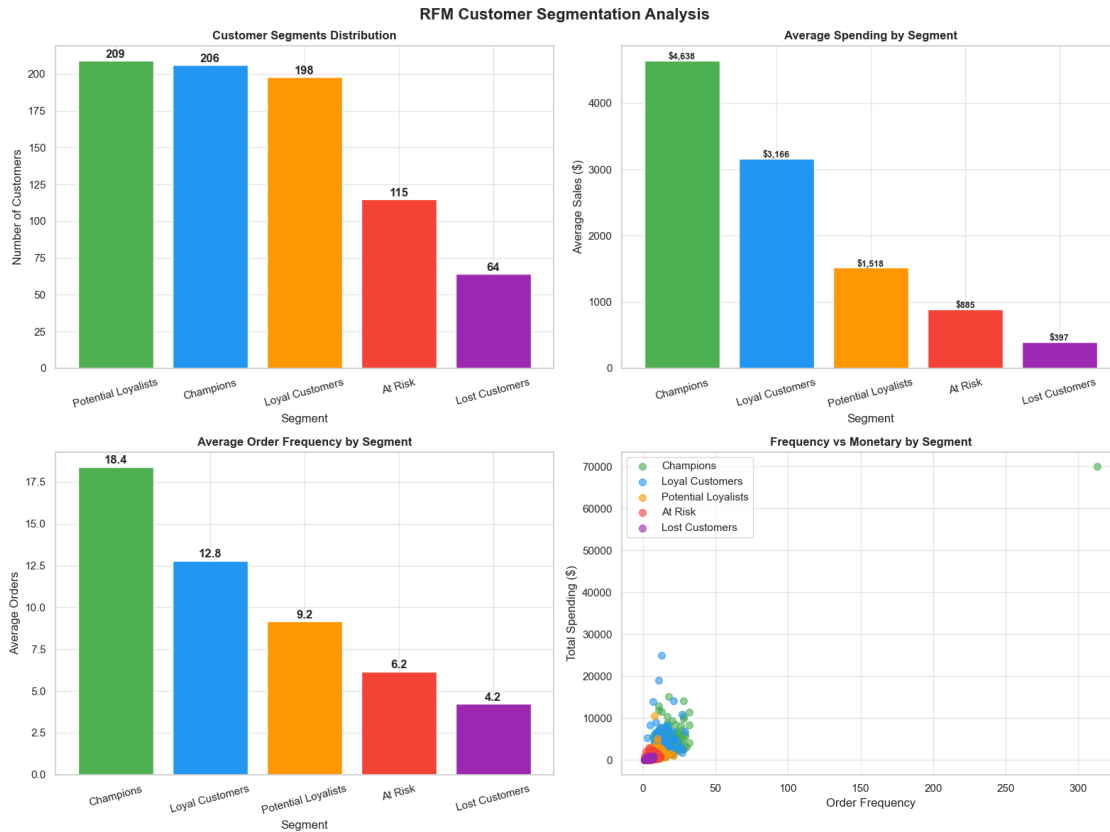
```

axes[1, 0].bar(segment_freq.index, segment_freq.values, color=colors)
axes[1, 0].set_title('Average Order Frequency by Segment', fontweight='bold')
axes[1, 0].set_xlabel('Segment')
axes[1, 0].set_ylabel('Average Orders')
axes[1, 0].tick_params(axis='x', rotation=15)
for i, v in enumerate(segment_freq.values):
    axes[1, 0].text(i, v + 0.2, f'{v:.1f}', ha='center', fontweight='bold')

# Chart 4 - RFM Scatter Plot
colors_map = {
    'Champions': '#4CAF50',
    'Loyal Customers': '#2196F3',
    'Potential Loyalists': '#FF9800',
    'At Risk': '#F44336',
    'Lost Customers': '#9C27B0'
}
for segment, color in colors_map.items():
    mask = rfm['Segment'] == segment
    axes[1, 1].scatter(rfm[mask]['Frequency'],
                      rfm[mask]['Monetary'],
                      c=color, label=segment,
                      alpha=0.6, s=50)
axes[1, 1].set_title('Frequency vs Monetary by Segment', fontweight='bold')
axes[1, 1].set_xlabel('Order Frequency')
axes[1, 1].set_ylabel('Total Spending ($)')
axes[1, 1].legend()

plt.suptitle('RFM Customer Segmentation Analysis', fontsize=16,
             fontweight='bold')
plt.tight_layout()
plt.savefig('chart13_rfm.png', dpi=150, bbox_inches='tight')
plt.show()
print(" RFM chart saved as 'chart13_rfm.png'")

```



RFM chart saved as 'chart13_rfm.png'